

Transformation of Non-Euclidean Space to Euclidean Space for Efficient Learning of Singular Vectors

SEUNGHYUN LEE[✉], (Associate Member, IEEE),
AND BYUNG CHEOL SONG[✉], (Senior Member, IEEE)

Department of Electronic Engineering, Inha University, Incheon 22212, South Korea

Corresponding author: Byung Cheol Song (bcsong@inha.ac.kr)

This work was supported in part by the Industrial Technology Innovation Program funded by the Ministry of Trade, Industry & Energy (MI, South Korea) (Development on Deep Learning based 4K30P Edge Computing enabled IP Camera System) under Grant 20006483, in part by the Institute of Information & Communications Technology Planning & Evaluation (IITP) funded by the Korea Government (MSIT) (Artificial Intelligence Convergence Research Center, Inha University) under Grant 2020-0-01389, and in part by the Industrial Technology Innovation Program through the Ministry of Trade, Industry, and Energy (MI, South Korea) (Development of Human-Friendly Human-Robot Interaction Technologies Using Human Internal Emotional States) under Grant 10073154.

ABSTRACT Singular value decomposition (SVD) is a popular technique to extract essential information by reducing the dimension of a feature set. SVD is able to analyze a vast matrix in spite of a relatively low computational cost. However, singular vectors produced by SVD have been seldom used in convolutional neural networks (CNNs). This is because the inherent properties of singular vectors such as sign ambiguity and manifold features make CNNs difficult to learn singular vectors. In order to overcome the limitations, this paper analyzes the undesirable properties of singular vectors and presents the transformation of singular vectors into Euclidean space as a smart solution. If the singular vectors are transformed to follow Euclidean geometry, SVD can be used for pooling to maintain the feature information well, which is called singular vector pooling (SVP). Since SVP can extract essential information from a feature map, it is robust against adversarial attacks in comparison to global average pooling. Thus, SVP shows a quantitative performance improvement of about 36% for the CIFAR10 dataset. In addition, we applied SVP to a knowledge distillation scheme that uses singular vectors in a restricted manner. As a result, SVP improved the performance by up to 1.7% for the CIFAR100 dataset.

INDEX TERMS Neural networks, pattern analysis, principal component analysis.

I. INTRODUCTION

Principal component analysis (PCA) is a technique used for the identification of a smaller number of uncorrelated variables known as principal components from a larger set of data. Several principal components have most information of a given data, so the dimension of the data can be significantly reduced or compressed by choosing only some of the principal components. Therefore, PCA is a beneficial technique for analyzing high-dimensional data. Among PCA techniques, singular value decomposition (SVD) is most popular because it can produce global solutions to various problems in computer vision, pattern recognition, and machine learning [1]–[3]. As the derivative function

of SVD was developed [4], SVD has been applied even to convolutional neural networks (CNNs) that can deal with very high-dimensional data. For instance, SVD was employed to obtain a unique square root of a symmetric positive definite (SPD) matrix for subspace analysis [5]–[7] or higher-order pooling [4], [8]–[10].

Previous works show that SVD can effectively extract essential information from feature maps. However, since singular vectors have undesirable properties such as sign ambiguity [11] and manifold features, they have been seldom studied to be directly learned in spite that they have feature map's essential information. In order to avoid undesirable properties, conventional approaches adopted an SPD matrix consisting of singular vectors. However, since they should expand the compressed feature's dimension again, they require additional computational and memory cost.

The associate editor coordinating the review of this manuscript and approving it for publication was Claudio Cusano[✉].

Lee *et al.* proposed a knowledge distillation method using SVD (KD-SVD) the harmful properties of singular vectors and defined singular vectors as knowledge [12]. But this algorithm has a limitation that it needs guidance from a teacher network.

In this paper, we define a singular vector as a feature vector and make its information directly learnable without re-composition and guidance. To do this, we analyze two problematic properties of singular vectors and proposes solutions to effectively remove the unfavorable properties based on the analysis. The sign ambiguity is eliminated so that singular vectors with similar information are gathered, and then singular vectors are converted from manifold to Euclidean space. Here, Euclidean geometry that is useful for ordinary learning schemes is employed. This procedure does not simply make singular vectors learnable but transform them to follow Euclidean geometry. Thus, SVD can be easily analyzed with layers based on Euclidean geometry.

Also, we propose a method called singular vector pooling (SVP), which is the only pooling method using SVD to our knowledge. SVP is basically robust to noise due to the nature of SVD. For example, in adversarial learning with the CIFAR10 dataset, SVP provides about 36% better performance than global average pooling (GAP). As shown in the feature distributions of Fig. 1, SVP has smaller intra-class variance and larger inter-class variance than GAP, which is a great advantage of SVP. In addition, when the SVP is applied to KD-SVD, which has learned singular vectors relatively naively, the performance of a student network can be improved by about 1.7% for the CIFAR100 dataset. The contribution of this paper is summarized as follows:

- This paper analyzes the properties of disturbing learning of singular vectors. Then, we propose skills to eliminate the unfavorable properties and present a method to transform singular vectors on non-Euclidean space to Euclidean space for effective learning.
- We propose SVP as a novel pooling method using SVD, and prove that applying SVP to CNN's feature maps is very useful for obtaining essential information.
- In addition, this paper shows that since singular vectors preserve feature map's information well, they are not only robust to adversarial attack, but also effective in knowledge distillation.

II. RELATED WORK

As one of the most popular PCA techniques, SVD is often used to reduce the dimension of a matrix or to extract essential information from a matrix or dataset. Even in the field of machine learning, the use of SVD is expanding more and more [4], [5], [8]–[10], [12]–[15]. For example, SVD has been applied to a typical CNN as the derivative of SVD has been mathematically derived recently. The most common example is the computation of a unique square root of an SPD matrix. This method is used for various purposes, such as

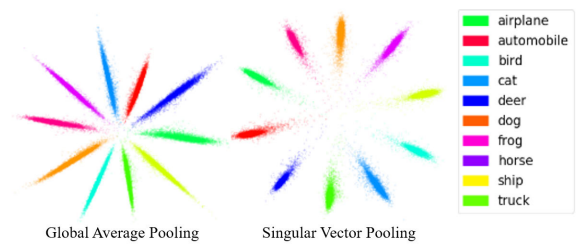


FIGURE 1. Comparison of feature vectors obtained by global average pooling and singular vector pooling.

covariance pooling [4], [8], [14], [15] and manifold embedding [5], [8]–[10]. In addition, there are several cases where a good property that singular vector(s) obtained from SVD has the compressed information is used for global pooling.

Since global pooling was first introduced in [16], it has evolved in various ways, such as employing trainable variables [17] and adopting non-linear functions [18], [19]. On the other hand, a global pooling method using SVD was proposed [20], which was based on covariance pooling. Covariance pooling obtains the covariance matrix $\mathbf{F}\mathbf{F}^T$ of a feature matrix $\mathbf{F}$ and extracts meaningful information from covariance. Reference [4] showed that if matrix logarithm is obtained by SVD, tangent space mapping is applicable. Recently, various covariance pooling methods have been proposed, inspired by [4]. For instance, [14] presented a method to solve the instability of SVD gradients, and [8] reduced the computational cost of SVD through Newton-Schulz iteration. Also, [15] showed that covariance pooling can be effectively applied in the middle of the network.

However, very few studies directly use singular vectors in deep learning. As far as we know, KD-SVD [12] may be the first deep learning technique that uses essential information of singular vectors directly. In KD-SVD, singular vectors are extracted from two feature maps by SVD, a correlation between two singular vectors is computed, and the correlation is distilled as knowledge. To facilitate the learning of singular vectors, KD-SVD uses a teacher network as a guide. However, such an approach is not applicable to general architectures other than the student-teacher network where the teacher network plays a guide role.

According to our survey, the use of SVD in deep learning is continuously being studied, but there is no way to learn singular vectors directly. On the other hand, it is also true that many methods using singular vectors for various computer vision tasks have been studied before the deep learning era [11], [21], [22]. We argue that the reason for this recent research trend is that although singular vectors have good information, there are some bad properties making difficult to learn them with common deep learning schemes. Therefore, by analyzing and removing these properties, we intend to propose a way that singular vectors can be utilized in deep learning scheme. Ultimately, the proposed algorithm will provide an opportunity to use SVD in various machine learning applications.

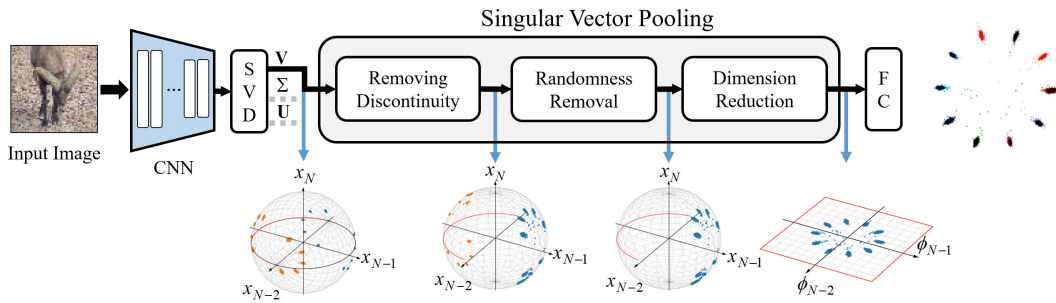


FIGURE 2. An example of CNN architecture using SVP. At the bottom of the figure, we show the process of transforming the singular vectors obtained through SVD prior to learning and the process of changing the feature distribution through each process.

III. METHODS

We theoretically analyze the undesirable properties that generally make the learning of singular vectors difficult. Based on the theoretical analysis, we propose SVP as a solution to overcome the limitations of using singular vectors. Fig. 2 shows the overall structure of SVP, which is a sort of post-processing for learning singular vectors. Feature maps obtained from CNN are first decomposed by SVD, and then the output singular vectors are transformed to facilitate learning by SVP. Finally, the transformed information is classified.

A. PROPERTIES OF SINGULAR VECTORS

Assume that a feature map obtained by a CNN has a spatial shape of $H \times W$ and a feature depth of D , and it can be transformed into a matrix form. Then, the feature map of matrix form can be interpreted as a set composed of HW D -dimensional vectors. We define this as $\mathbf{F}$. Decomposing $\mathbf{F}$ using SVD results in:

$$\mathbf{F} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \quad (1)$$

Left-hand matrix $\mathbf{U}$ has feature specific information, right-hand matrix $\mathbf{V}$ has global feature information, and the central matrix $\mathbf{\Sigma}$ indicates singular value magnitudes [1]. Since the singular values are decomposed in descending order, SVD of the same matrices always yields the same $\mathbf{\Sigma}$. On the other hand, $\mathbf{U}$ and $\mathbf{V}$ are created in pairs as follows:

$$\mathbf{F} = \sum_{k=1}^K (p_k \mathbf{u}_k) \sigma_k (p_k \mathbf{v}_k^T), p_k \in \{1, -1\} \quad (2)$$

where

$$\mathbf{U} = [\mathbf{u}_k]_{1 \leq k \leq K}, \mathbf{V} = [\mathbf{v}_k]_{1 \leq k \leq K}, K = \min(HW, D) \quad (3)$$

Eq. (2) indicates that singular vectors with identical information can be randomly distributed in two different positions due to random variable p_k . This phenomenon is called sign ambiguity [11]. This is an undesirable property that must be removed because it makes learning singular vectors difficult. Meanwhile, each singular vector is a unit vector with a norm of 1. In other words, singular vectors are the points on a unit hypersphere—i.e., a manifold. Therefore, learning singular

vectors with a generic neural network and Euclidean geometry can be very inefficient or impossible. This is the second problem. In summary, singular vectors have two fundamental problems as follows.

- Singular vectors with similar information are randomly distributed in two areas due to sign ambiguity.
- It is inefficient to learn singular vectors on manifold space through a general neural network scheme.

B. OVERVIEW OF THE PROPOSED METHOD

We first describe a solution for learning singular vectors with the manifold property. Since the manifold is a unit hypersphere, it can be mapped to Euclidean space by disentangling each coordinate through a specific coordinate conversion. As a result, it becomes possible to use SVD as a layer of CNN because it is easier to learn singular vectors based on Euclidean geometry and to transform them. (see Sec. III-E).

However, since each coordinate in the spherical space is bounded, the singular vector distribution can be discontinuous near the boundary. This phenomenon can negatively affect the feature embedding of CNN. Therefore, rotation is applied so that the singular vectors are not located near the boundary before coordinate conversion. (see Sec. III-C).

Next, we present a method to remove the sign ambiguity phenomenon. Suppose that singular vectors are distributed in two regions around a specific center vector due to sign ambiguity. Similarity of each singular vector is calculated based on the center vector. By reversing the sign of a singular vector with negative similarity, the sign ambiguity can be resolved. This method is very simple and efficient as in Sec. III-D.

On the other hand, it is not easy to determine an accurate center vector because the singular vector distribution can change as learning progresses. Thus, we propose a way to approximate the center vector during learning. This process is performed after coordinate conversion for convenient calculation. (see Sec. III-F)

C. REMOVING DISCONTINUITY BY ROTATION

Mapping of singular vectors in the spherical coordinate system is equivalent to converting them from non-Euclidean space to Euclidean space. However, when singular vectors

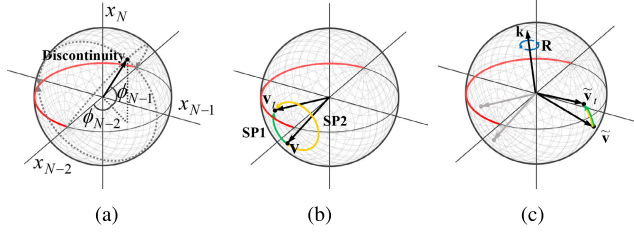


FIGURE 3. Discontinuity and rotation effects in the spherical coordinate system. Red, green, yellow, and blue lines indicate a discontinuity boundary, SP1, SP2, and rotation, respectively. (a) Discontinuity in spherical coordinates (b) SP1 and SP2 before rotation (c) SP1 and SP2 after rotation.

are directly mapped to spherical coordinates, a discontinuity problem may occur.

Let $(r, \phi_1, \phi_2, \dots, \phi_{N-1})$ denote N -dimensional spherical coordinates. Each component of $(\phi_1, \phi_2, \dots, \phi_{N-2})$ has a bounded range of $[0, \pi]$, and ϕ_{N-1} is limited to $[-\pi, \pi]$, which means that discontinuity may occur at the boundary of each component, as shown in Fig. 3(a). For convenience, the three-dimensional space is assumed in the figure. If two feature vectors $\mathbf{v}$ and $\mathbf{v}_t$ are present near the discontinuity boundary, as in Fig. 3(b), we can consider two different shortest paths: one through the discontinuity boundary (SP1) and another to avoid the discontinuity boundary (SP2). Therefore, the learning efficiency may be reduced because singular vectors can practically move to longer paths such as SP2 rather than the real shortest path (SP1) during the learning process. Thus, singular vectors should be kept as distant from the discontinuity boundary as possible. In other words, the singular vector distribution must be intentionally centered in the spherical coordinate system. The center of the spherical coordinate system $\{\frac{\pi}{2}, \frac{\pi}{2}, \dots, \frac{\pi}{2}, 0\}$ is converted to $\{0, \dots, 0, 1, 0\}$ of the Cartesian coordinate using spherical to Cartesian coordinates conversion equation in [23].

As a result, the center of the singular vector distribution $\mathbf{v}_c$ must be rotated so that it becomes the coordinate center, $\tilde{\mathbf{v}}_c$ which is $\{0, \dots, 0, 1, 0\}$. How to obtain $\mathbf{v}_c$ will be given in Sec. III-F. Then, the rotation matrix $\mathbf{R}$ is computed by using the well-known Rodrigues rotation as follows:

$$\mathbf{k} = \mathbf{v}_c \times \tilde{\mathbf{v}}_c, \quad \theta = \arccos(\mathbf{v}_c \cdot \tilde{\mathbf{v}}_c) \quad (4)$$

$$\mathbf{K} = [\mathbf{k}]_{\times} = \frac{\mathbf{v}_c \tilde{\mathbf{v}}_c^T - \tilde{\mathbf{v}}_c \mathbf{v}_c^T}{\sin \theta} \quad (5)$$

$$\mathbf{R} = \mathbf{I} + (\sin \theta) \mathbf{K} + (1 - \cos \theta) \mathbf{K} \cdot \mathbf{K} \quad (6)$$

where $\mathbf{k}$ is the direction of the rotation axis. Using the rotation matrix of Eq. (6), $\tilde{\mathbf{V}}$ is obtained by:

$$\tilde{\mathbf{V}} = \mathbf{R} \mathbf{V} \quad (7)$$

We can obtain $\tilde{\mathbf{U}}$ by applying the same process to $\mathbf{U}$.

As shown in Fig. 3(c), the shortest path and the real path now coincide. However, if singular vector variance is too high, some singular vectors still exist near the discontinuity boundary. Fortunately, SVD computes a least-square solution which is not sparse, and regularization skills such as weight

decay and batch normalization [24] also control feature map's variance to some extent. So, we can ignore this high variance problem. Therefore, the discontinuity problem is eliminated so that a singular vector can be effectively learned. However, the two problems of singular vectors mentioned in Section 3.1 still remain unsolved.

D. SIGN AMBIGUITY REMOVAL IN SINGULAR VECTORS

$\mathbf{U}$ and $\mathbf{V}$ in Eq. (1) exist as a pair, and one of them is used as a reference. We adopt an approach of aligning $\tilde{\mathbf{V}}$ based on $\tilde{\mathbf{U}}$ as a reference. Let $\tilde{\mathbf{u}}_k$ be the k -th singular vector. Then $\tilde{\mathbf{u}}_k$ and $-\tilde{\mathbf{u}}_k$ exist concurrently with the same information but opposite signs. Therefore, a domain in which a singular vector exists can be divided into a positive half-sphere that is decomposed in the forward direction and a negative half-sphere that is decomposed in the reverse direction. Thus, we eliminate the randomness property by multiplying a singular vector on the negative half-sphere by -1 . In the previous section, the center vector of the singular vector distribution is rotated to $\tilde{\mathbf{u}}_c$, which is $\{0, 0, \dots, 0, 1, 0\}$. So we can verify the hyper-sphere where $\tilde{\mathbf{u}}_k$ is in according to the sign of $\tilde{u}_{HW-1,k}$. Finally, $\hat{\mathbf{U}}$ and $\hat{\mathbf{V}}$ that are arranged based on $\tilde{\mathbf{u}}_c$ are defined as follows:

$$\hat{\mathbf{U}} = [\text{sign}(\tilde{u}_{HW-1,k}) \tilde{\mathbf{u}}_k]_{1 \leq k \leq K} \quad (8)$$

$$\hat{\mathbf{V}} = [\text{sign}(\tilde{u}_{HW-1,k}) \tilde{\mathbf{v}}_k]_{1 \leq k \leq K} \quad (9)$$

Since the sign ambiguity phenomenon in $\hat{\mathbf{U}}$ and $\hat{\mathbf{V}}$ has been removed, $\hat{\mathbf{U}}$ and $\hat{\mathbf{V}}$ are easier to analyze than the original $\tilde{\mathbf{U}}$ and $\tilde{\mathbf{V}}$.

E. COORDINATE CONVERSION FOR EFFICIENT LEARNING

To convert the coordinate system of $\hat{\mathbf{V}}$, it is rewritten as:

$$\hat{\mathbf{V}} = [\hat{v}_{n,k}]_{1 \leq n \leq D, 1 \leq k \leq K} \quad (10)$$

Then, the D -dimensional transformation of $\hat{\mathbf{V}}$ in Cartesian coordinates into spherical coordinates is defined as follows:

$$\bar{\mathbf{V}} = [\bar{v}_{n,k}]_{1 \leq n \leq D-1, 1 \leq k \leq K} \quad (11)$$

where

$$\bar{v}_{n,k} = \begin{cases} \text{sign}(\hat{v}_{D,k}) \arccos\left(\frac{\hat{v}_{n,k}}{S_{n,k}}\right), & \text{if } n = D-1 \\ \arccos\left(\frac{\hat{v}_{n,k}}{S_{n,k}}\right), & \text{otherwise} \end{cases} \quad (12)$$

$$S_{n,k} = \sqrt{\sum_{s=n}^D \hat{v}_{s,k}^2 + \epsilon} \quad (13)$$

In Eq. (13), ϵ is a constant to prevent zero-division and is set to 10^{-3} . So, $\bar{\mathbf{V}}$ on the spherical coordinate system is obtained. However, the 1st derivative of the arccosine in Eq. (12) can easily diverge, as shown in Eq. (14).

$$\frac{d}{dz} \arccos(z) = -\frac{1}{\sqrt{1-z^2}}, \quad (-1 < z < 1) \quad (14)$$

So, we approximate the arccosine to $-z + \frac{\pi}{2}$ by the first order Taylor expansion. Next, we add a constant term for keeping a zero-centered feature. Finally, Eq. (12) is re-written by

$$\bar{v}_{n,k} \cong \begin{cases} \text{sign}(\hat{v}_{D,k}) \left(-\frac{\hat{v}_{n,k}}{S_{n,k}} + 1 \right), & \text{if } n = D - 1 \\ -\frac{\hat{v}_{n,k}}{S_{n,k}}, & \text{otherwise} \end{cases} \quad (15)$$

As a result, $\hat{\mathbf{V}}$ is converted to $\bar{\mathbf{V}}$ that can be learned by a generic neural network. At the same manner, $\bar{\mathbf{U}}$ is derived.

F. ESTIMATION OF THE CENTER VECTOR

In this section, the process of estimating the center vector mentioned in Sec. III-C is described. Since this process has to be explained in connection with Sec. III-D, it is described with $\mathbf{u}_c$ instead of $\mathbf{v}_c$. The proposed method rotates $\mathbf{u}$ around $\mathbf{u}_c$, moves the center of the distribution to $\{0, 0, \dots, 0, 1, 0\}$, and determines which half-sphere it belongs to through the sign of $\hat{u}_{HW-1,k}$. If $\mathbf{u}_c$ is inaccurate, this process will be erroneous. Thereby, the sign ambiguity problem cannot be solved properly. To observe this phenomenon, the distribution of $\bar{\mathbf{u}}$ obtained from $\mathbf{u}_c$ is shown in the first row of Fig. 4. Here $\mathbf{u}_c$ is experimentally set to a unit vector with all components of the same size. We can find that the distributions of the two half-spheres are not aligned correctly because of the sign ambiguity problem caused by inaccurate $\mathbf{u}_c$. To obtain the precise $\mathbf{u}_c$, we suggest learning of $\mathbf{u}_c$. Since we are hard to understand the statistical characteristics of singular vectors due to two inherent bad properties, we learn $\mathbf{u}_c$ based on $\bar{\mathbf{u}}$. Note that each component of features produced by neural networks follows the Gaussian distribution in general. So, assuming that each component of the aligned $\bar{\mathbf{u}}$ has the Gaussian distribution, $\mathbf{u}_c$ is learned through KL-divergence [25] of Eq. (16) so that the assumption is correct.

$$L_{\bar{\mathbf{u}},k}^{KL} = \sum_{k=1}^K \sum_b^B \text{Norm}(\mu_{\bar{\mathbf{u}},k}, \sigma_{\bar{\mathbf{u}},k}^2) \log \frac{\text{Norm}(\mu_{\bar{\mathbf{u}},k}, \sigma_{\bar{\mathbf{u}},k}^2)}{p(\bar{\mathbf{u}},k, \mathbf{u}_c)} \quad (16)$$

where $\text{Norm}(\mu_{\bar{\mathbf{u}},k}, \sigma_{\bar{\mathbf{u}},k}^2)$ indicates a Gaussian distribution. Furthermore, $\sigma_{\bar{\mathbf{u}},k}^2 = \text{var}(\bar{\mathbf{u}},k)$, and mean $\mu_{\bar{\mathbf{u}},k}$ is empirically set to 0. B is the batch size. As shown in Fig. 4, in order to minimize $L_{\bar{\mathbf{u}},k}^{KL}$, two distributions of $\bar{\mathbf{u}}$ get merged and form a single Gaussian distribution, which means that the sign ambiguity problem is solved. Therefore, the proposed method can estimate $\mathbf{u}_c$ overcoming two bad properties of singular vectors. In the same manner, $\mathbf{v}_c$ is estimated by minimizing $L_{\bar{\mathbf{v}},k}^{KL}$. Note that $L_{\bar{\mathbf{u}},k}^{KL}$ and $L_{\bar{\mathbf{v}},k}^{KL}$ are loss functions for learning $\mathbf{u}_c$ and $\mathbf{v}_c$ only and do not affect the network parameters.

IV. EXPERIMENTAL RESULTS

We performed three experiments to verify the proposed method. First, we evaluated the performance of SVP in a CNN structure when using SVD. Because classification task outputs only class information, we use one singular vector

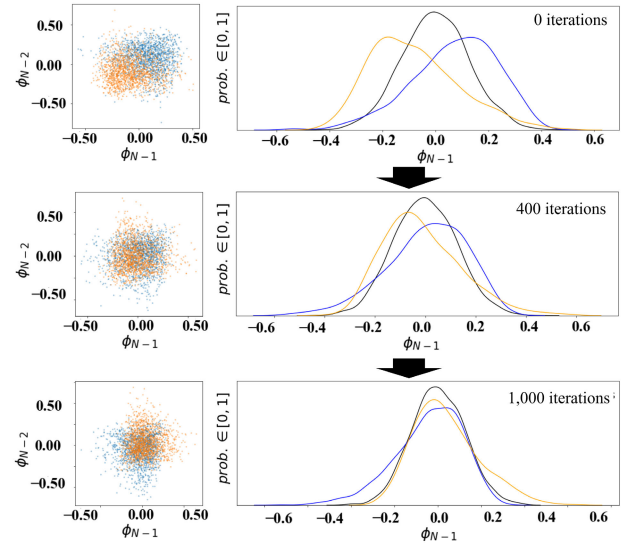


FIGURE 4. Learning process of singular vectors which are converted to the spherical coordinate. The left column is plotting the last two components and the right column is the distribution. The black line shows a Gaussian distribution, and the blue and yellow lines indicate the distributions of singular vectors in two half-spheres obtained by applying the proposed method. During the learning process, it gradually gets closer to the Gaussian distribution, indicating that the basis is well learned.

for the classification task. Second, we applied the proposed method to KD-SVD, which is a representative network using singular vectors. Since KD-SVD compresses CNN's feature maps with several dominant singular vectors, the performance of the student network improves. In this experiment, we additionally show an effect of using multiple singular vectors for extracting important information from feature maps. Third, the performance of the proposed method is analyzed through an ablation study.

A. EXPERIMENT ENVIRONMENT

The datasets used in the following experiments are CIFAR10, CIFAR100 [26], Tiny-ImageNet, and ImageNet2012 [27], which are all normalized to have values of $[-0.5, 0.5]$. CIFAR10 and CIFAR100 have 10 and 100 labels, respectively, and are compact datasets consisting of 50,000 color images of 32×32 pixels. Because these datasets require relatively little cost, they are used for basic verification of the proposed method. Tiny-ImageNet is a medium-sized dataset with 100,000 color images of 64×64 pixels and has 200 labels. ImageNet2012 is a large dataset with more than 1.5 million high-resolution color images and 1,000 labels. The last two datasets are used to prove that the proposed method works well even for huge datasets with large resolutions.

A particular augmentation is applied to each dataset. Horizontal random flip is applied to all the datasets. Also, the images of CIFAR10 and CIFAR100 are zero-padded by 4 pixels, and the images of Tiny-ImageNet [27] are zero-padded by 8 pixels. Then, the zero-padded images are randomly cropped to the original size. In case of ImageNet2012, all

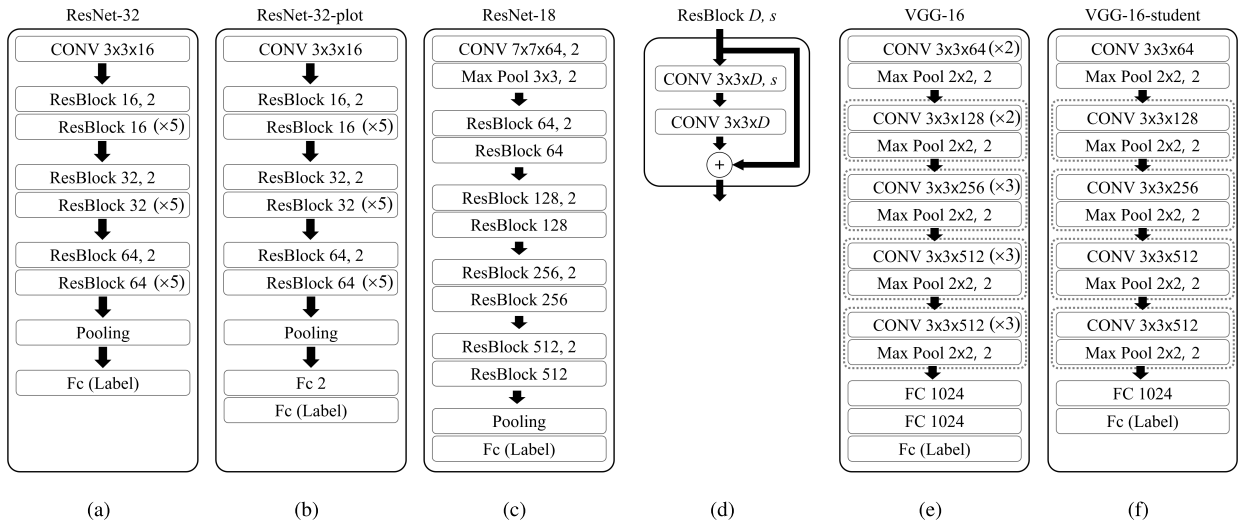


FIGURE 5. The block diagrams of the networks introduced in this paper.

the images are resized to 256×256 , and then the training image are produced by randomly cropping the resized images to 224×224 . The test set is constructed by cutting the central area of each image to 224×224 .

All algorithms are implemented using Tensorflow [28], and their network architectures are shown in Fig. 5. ‘CONV’ and ‘Max Pool’ indicate convolutional layer and max pooling layer, respectively. ‘FC’ stands for fully connected layer. Also $H \times W$ is the kernel size, D is the output depth. s means stride and the same layer repeats by N . (Label) indicates the number of labels in the target dataset. The activation function for convolution layers and all FC layers except the end of the network is ReLU [29]. In Fig. 5(a), 5(b) and 5(c), one of SVP, global average pooling (GAP) [16], global max pooling (GMP) [18], and matrix power normalized covariance pooling (MPN) [8] is adopted for pooling block. Also, in Fig. 5(a), 5(b), 5(c) and 5(d), batch normalization [24] is used next to the convolutional layer. The dotted gray boxes in Fig. 5(e) and 5(f) are the layer modules for knowledge distillation. The input and output feature maps of this module are sensed and applied to both the authentic KD-SVD [12] and the KD-SVP where SVP was applied to KD-SVD.

The weights of all networks are initialized with He’s initialization [30], and L_2 regularization is applied. A stochastic gradient descent (SGD) [31] is used as the optimizer, and a Nesterov accelerated gradient [32] is applied. All numerical values in the tables and figures are the averages of all five trials.

B. EVALUATION OF SINGULAR VECTOR POOLING

Most CNNs increase the size of the receptive field and reduce the size of the feature map through pooling. GMP and GAP are the most popular pooling methods, but both methods suffer from the inherent loss of information. SVP can improve the performance of a given network because it can reduce

the size of feature maps while keeping as much of their information as possible.

The base network used for this experiment is ResNet-32 [30] which described in Fig. 5(a). We analyzed the results by replacing GAP of ResNet-32 with GMP [18] and SVP, respectively. In case of CIFAR10, learning is proceeded for 200 epochs and the initial learning rate is equal to 10^{-2} , which is reduced by 0.1 times at 100 and 150 epochs. The batch size is set to 128, and the weight decay of L_2 regularization is fixed to 10^{-4} . ImageNet2012 is trained for 90 epochs, and the initial learning rate is set to 10^{-2} , and the learning rate is reduced by 0.1 times at 30 and 60 epochs. The batch size in this experiment is 128 and the weight decay of L_2 regularization is fixed to 5×10^{-4} .

TABLE 1. The comparison of the proposed SVP with GAP and GMP for CIFAR10 dataset. ‘ACC’ indicates accuracy, ‘Silh’ indicates silhouette score, and T indicates forward time for each pooling layer.

Pooling Methods	Training dataset		Test dataset		T[ms]
	Acc.[%]	Silh	Acc.[%]	Silh	
GAP	99.88	0.393	89.87	0.295	12.4
	± 0.01	± 0.002	± 0.26	± 0.002	
GMP	99.78	0.327	89.43	0.246	12.2
	± 0.04	± 0.004	± 0.27	± 0.004	
MPN	99.86	0.346	90.53	0.270	14.1
	± 0.02	± 0.002	± 0.05	± 0.001	
SVP	99.91	0.837	90.54	0.613	13.2
	± 0.02	± 0.003	± 0.17	± 0.002	

Table 1 shows the experimental results for CIFAR10. We can find the validation accuracy of SVP is 0.67% and 0.91% better than that of GAP and GMP, respectively. However, this is not a big improvement and almost same with MPN. Nevertheless, the silhouette score [33] of SVP on training dataset is better by 0.444, 0.510, and 0.551 than that of GAP, GMP, and MPN, respectively. SVP shows excellent silhouette scores that are more than twice those of the

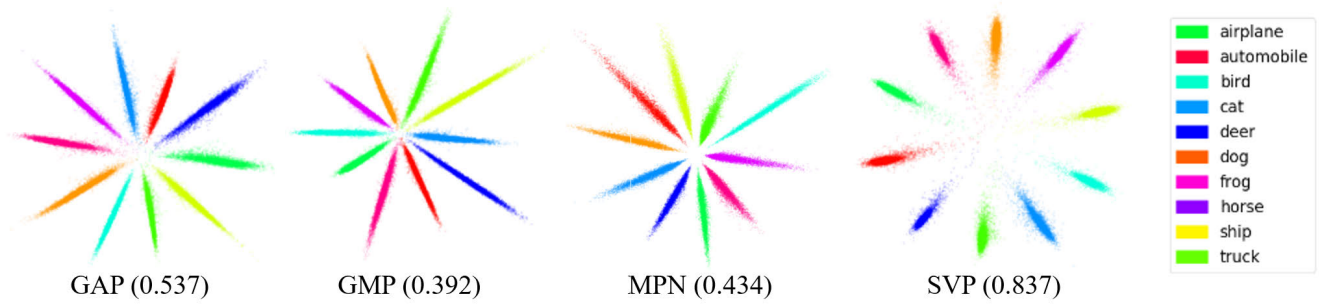


FIGURE 6. The distribution of feature vectors obtained by several pooling techniques for CIFAR10 training set.

other pooling methods. This indicates that SVP makes feature vectors get a high inter-class variance and a low intra-class variance. Also, the proposed method has a lower forward time than MPN, which is a light-weight second-order pooling method. Therefore, we can find that the proposed method can provide better clustering performance with less computation time.

Also, to illustrate the clustering performance clearly, we performed the same experiment through a network with an additional FC layer having a two-dimensional output as shown in Fig. 5(b). The network is learned for CIFAR10 dataset until overfitting occurs in order to discriminate clusters clearly. The feature distributions are shown in Fig. 6. The feature distributions of GAP, GMP, and MPN have very large intra-class variation, and the features tend to gather near the origin. That is, their inter-class variation is low. However, the feature distribution of SVP shows better separation than those of GAP, GMP, and MPN. This means that SVP makes a given network operate more robustly to noise and attack. In addition, we show the evaluation results for ImageNet2012 through ResNet-18 of Fig. 5(c) to generalize the proposed method (see Table 2). We could observe a trend similar to the results in CIFAR10 dataset, which means that SVP extracts essential information well even with large feature maps.

TABLE 2. The comparison of SVP with GAP for ImageNet2012 dataset.

Pooling Methods	Training dataset		Test dataset	
	Acc. [%]	Silh	Acc. [%]	Silh
GAP	84.87	-0.013	64.60	0.001
	± 0.11	± 0.006	± 0.36	± 0.005
SVP	84.09	0.004	64.76	0.031
	± 0.08	± 0.005	± 0.24	± 0.003

C. ROBUSTNESS AGAINST ADVERSARIAL ATTACKS

We next examined the robustness of SVP against adversarial attacks. The attack methods include the fast gradient sign method (FGSM) [34] and basic iterative method (BIM) [35]. FGSM is a gradient-based adversarial attack and one of the simplest and most effective attack methods. The image X^{adv} generated by FGSM is defined by:

$$X^{adv} = X + \alpha \text{sign}(\nabla_X J(X, y_{true})) \quad (17)$$

where X is an input image, y_{true} is the label, and J stands for the Jacobian. The attack rate α is set to 0.01 in this experiment.

Since BIM performs FGSM iteratively, BIM generally attacks more strongly than FGSM. The I -th image generated by BIM is defined by Eqs. (18) and (19):

$$X_0^{adv} = X \quad (18)$$

$$X_I^{adv} = \text{Clip}_{X,\beta} \left\{ X_{I-1}^{adv} + \alpha \text{sign} \left(\nabla_X J \left(X_{I-1}^{adv}, y_{true} \right) \right) \right\} \quad (19)$$

where β is the clipping threshold. The following values are used in this experiment: $\alpha = 0.01$, $\beta = 0.002$, and $T = 5$. All hyper-parameters are set as in Sec. IV-B.

TABLE 3. Experimental result on white-box adversarial attack. The first three rows correspond to natural training, and the second three rows are the result of adversarial training.

Training Scheme	Attack Methods	Pooling Methods [%]			
		GAP	GMP	MPN	SVP
Natural	None	89.87	89.43	90.53	90.54
		± 0.26	± 0.27	± 0.05	± 0.17
		22.73	30.62	24.59	58.80
	FGSM	± 0.69	± 0.93	± 0.75	± 0.39
		3.40	9.48	3.77	41.80
		± 0.57	± 1.28	± 0.36	± 0.66
Adversarial	None	86.32	85.75	86.04	86.89
		± 0.33	± 0.12	± 0.07	± 0.17
		61.18	60.59	63.94	64.28
	FGSM	± 0.29	± 0.27	± 0.22	± 0.30
		58.41	57.50	59.83	61.14
		± 0.21	± 0.29	± 0.14	± 0.23

To compare the results of Sec. IV-B, the CIFAR10 dataset was employed in this experiment, using ResNet-32 in Fig. 5(a) and ResNet-32-plot in Fig. 5(b). Table 3 shows the white-box attack results for several pooling methods. For FGSM with natural and adversarial training, SVP shows 36.07%, 28.18%, and 34.21% higher accuracies than GAP, GMP, and MPN, respectively. Overall, SVP provides the best performance for all other cases. The performance improvement is interpreted through the high silhouette score of the feature vector obtained by SVP. Fig 7 plots the feature vector distributions by employing the network used in Fig. 6. In the other pooling methods, the clusters are scattered due to the attack, but SVP maintains a relatively strong inter-cluster

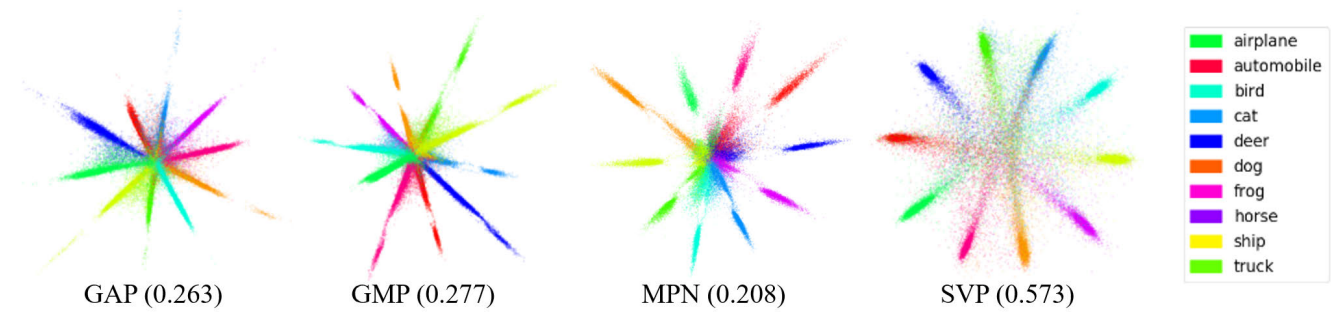


FIGURE 7. The distribution of feature vectors obtained by several pooling techniques for an adversarial attacked CIFAR10 training set.

TABLE 4. Experimental results on black-box adversarial attack. The first three rows represent the natural training, and the second three rows represent the adversarial training results.

Training Scheme	Attack Methods	Pooling Methods [%]			
		GAP	GMP	MPN	SVP
Natural	None	89.87	89.43	90.53	90.54
		± 0.26	± 0.27	± 0.05	± 0.17
		69.14	69.50	69.43	70.10
	FGSM	± 0.50	± 0.96	± 0.58	± 0.61
		64.77	65.07	65.03	65.70
		± 0.60	± 0.51	± 0.67	± 0.92
Adversarial	None	86.32	85.75	86.04	86.89
		± 0.33	± 0.12	± 0.04	± 0.17
		83.18	82.53	82.87	83.62
	FGSM	± 0.21	± 0.16	± 0.24	± 0.12
		83.44	82.78	83.51	83.79
		± 0.21	± 0.15	± 0.23	± 0.12

distance. Furthermore, Fig. 8 shows the performance changes according to the attack rates. As the attack rate increases, the gaps between SVP and the other pooling methods increase. For example, when α is 0.02, SVP shows 40.99%, 33.60%, and 40.62% better than GAP, GMP, and MPN, respectively.

A black-box attack is a method of training and testing a target network using attacked data generated by an arbitrary network. The network used for attacking is VGG-16 [36], and the experimental results are shown in Table 4. SVP still outperforms GAP, GMP, and MPN, but the gap is somewhat reduced. This occurs because SVP is located at the end of the network and it is difficult for it to extract essential information from a feature map that has already been destroyed. The experiments with adversarial attacks prove that the proposed method is robust to noise. Despite the limitations, the proposed method makes the given network robust against noise without requiring additional techniques for singular vectors.

D. KNOWLEDGE DISTILLATION USING SVP

The feature vectors obtained by KD-SVP are easier to analyze than original singular vectors. In KD-SVD, the sign ambiguity is removed by aligning the singular vectors of the student network based on the singular vector of the teacher network. In addition, KD-SVD learns only similar features to learn the manifold features and defines the correlation by a radial basis function (RBF), which has smaller values as the distance becomes larger. In other words, singular vectors

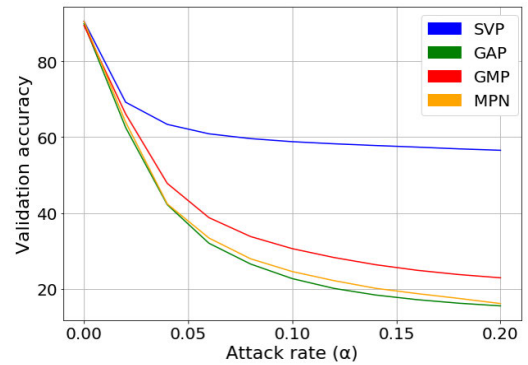


FIGURE 8. The performance change according to the intensity of adversarial attack.

are post-processed in a very naive manner in KD-SVD. Therefore, if SVP plays a post-processing role, the learning accuracy will improve. The correlation between feature vectors obtained by SVP is computed by RBF, similarly to KD-SVD. The CIFAR100 and Tiny-ImageNet are used in this experiment. The teacher network is VGG-16, and the student network is a condensed network that uses only one convolution of the same depth as VGG-16, which are described in Fig. 5(e) and 5(f). KD-SVD and KD-SVP are all learned with the same hyper-parameters. Their learning progressed by 200 epochs, with an initial learning rate of 10^{-2} and a reduction of 0.1 in 100 and 150 epochs. Also, the batch size is set to 128 and the weight decay of L_2 regularization is 10^{-4} .

TABLE 5. Performance of KD-SVP according to the number of singular vectors.

Methods	CIFAR-100 Accuracy [%]	Tiny-ImageNet Accuracy [%]
Teacher	67.99	55.16
Student	64.70 ± 0.21	53.51 ± 0.21
KD-SVD	66.54 ± 0.53 (+1.84)	55.10 ± 0.19 (+1.49)
KD-SVP	68.25 ± 0.06 (+3.53)	55.97 ± 0.20 (+2.46)

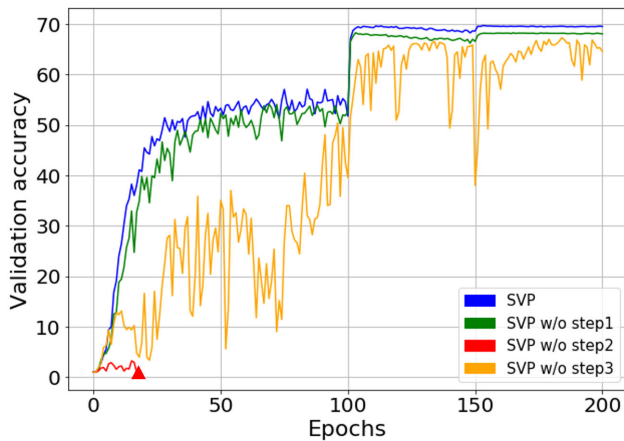
Table 5 shows that when the proposed method is applied to KD-SVD, the performance is improved by 1.69% for CIFAR100 and 0.97% for Tiny-ImageNet. This experiment shows that singular vectors are learned much better by KD-SVP. Thus, the experiment demonstrates that the

TABLE 6. Performance of KD-SVP according to the number of singular vectors.

Dataset	1	2	4
CIFAR-100	68.25 $\pm$ 0.06	68.96 $\pm$ 0.05	69.42 $\pm$ 0.05
Tiny-ImageNet	55.97 $\pm$ 0.20	56.24 $\pm$ 0.15	56.39 $\pm$ 0.17

TABLE 7. Performance change whenever each step of SVP is removed.

Pooling Methods	Training dataset		Validation dataset	
	Acc. [%]	Silh	Acc. [%]	Silh
SVP	98.37 $\pm$ 0.21	0.121 $\pm$ 0.16	69.50 $\pm$ 0.24	0.042 $\pm$ 0.12
SVP w/o step1	98.71 $\pm$ 0.17	0.082 $\pm$ 0.002	68.04 $\pm$ 0.44	0.026 $\pm$ 0.002
SVP w/o step2	N/A	N/A	N/A	N/A
SVP w/o step3	84.37 $\pm$ 9.98	0.102 $\pm$ 0.045	64.49 $\pm$ 4.71	0.037 $\pm$ 0.020

**FIGURE 9.** Performance comparison whenever removing each step of SVP. Red triangle means NaN.

application of SVP significantly improves the performance of a given network. Next, we analyze the performance of KD-SVP according to the number of used singular vectors. Table 6 shows that since more singular vectors give more information, they improve the performance of the student network.

E. ABLATION STUDY

The effect of each part of the proposed SVP on the overall performance of a given network is analyzed. SVP consists of rotation (step 1), removing sign ambiguity (step 2), and coordinate conversion (step 3). The model used for this experiment is ResNet-18, and CIFAR100 is chosen as the dataset. We measured the performance changes by removing each part one by one. Table 7 shows the results, and Fig. 9 shows the training plots.

Of course, whenever each step is omitted, the overall performance deteriorates. If step 1 is omitted (rotation), the sign ambiguity cannot be eliminated accurately because the singular vectors on the two half-spheres are not superimposed on each other. The performance decreases by about 1.46% because the error increases as the distance from the center of the coordinates deviates. However, the performance degradation is smaller than when omitting other steps. When

the coordinate conversion step is removed, manifold features are not sufficiently learned, which resulted in performance degradation by 5.01%. This occurred because singular vectors that are defined in non-Euclidean space are hard to learn by Euclidean geometry. Finally, if the sign ambiguity removal of step 2 is absent, learning is impossible. This experiment shows that each step of SVP is indispensable for proper learning of singular vectors.

V. DISCUSSION

One advantage of the proposed method is that converting singular vectors into Euclidean space enables to use Euclidean geometry. The merit makes a generic CNN learn the singular vectors. As another advantage, the proposed SVP is superior to other techniques that treat singular vectors as manifold features in terms of scalability. SVP outperforms conventional pooling methods such as GAP and GMP, and also it is more robust against noise. In addition, SVP is experimentally proven to provide further performance enhancement when it is applied to an existing knowledge distillation scheme such as [17]. On the other hand, SVP has a disadvantage that its computational complexity is somewhat higher than that of general pooling methods. Also, SVD requires burdensome computations, so it may be difficult to apply it directly to an embedded system or a mobile environment.

VI. CONCLUSION

SVD is one of the most important techniques in many areas of high-level data manipulation. Therefore, if SVD can be used in deep learning methods such as CNN, it could be useful for various purposes. However, singular vectors decomposed by SVD have not been widely used in CNNs yet, as they are generally difficult to handle. This study presented a starting point for effectively using singular vectors with essential information in deep learning. Although the proposed SVP has a simple form, it is very useful because it is about 36% more robust to adversarial attacks than GAP and produces 1.69% more improvement in knowledge distillation than the naïve solution [12]. Future work should be done to improve the performance of CNNs further by devising ways of more effectively using the information of singular vectors.

REFERENCES

- [1] O. Alter, P. O. Brown, and D. Botstein, "Singular value decomposition for genome-wide expression data processing and modeling," *Proc. Nat. Acad. Sci. USA*, vol. 97, no. 18, pp. 10101–10106, Aug. 2000.
- [2] E. L. Denton, W. Zaremba, J. Bruna, Y. LeCun, and R. Fergus, "Exploiting linear structure within convolutional networks for efficient evaluation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 1269–1277.
- [3] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Incremental singular value decomposition algorithms for highly scalable recommender systems," in *Proc. 5th Int. Conf. Comput. Inf. Sci.*, 2002, pp. 27–28.
- [4] C. Ionescu, O. Vantzos, and C. Sminchisescu, "Matrix backpropagation for deep networks with structured layers," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 2965–2973.
- [5] Z. Huang, J. Wu, and L. Van Gool, "Building deep networks on Grassmann manifolds," 2016, *arXiv:1611.05742*. [Online]. Available: <http://arxiv.org/abs/1611.05742>
- [6] R. Ranftl and V. Koltun, "Deep fundamental matrix estimation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Sep. 2018, pp. 284–299.

- [7] S. Suwajanakorn, N. Snavely, J. J. Tompson, and M. Norouzi, "Discovery of latent 3d keypoints via end-to-end geometric reasoning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 2059–2070.
- [8] P. Li, J. Xie, Q. Wang, and Z. Gao, "Towards faster training of global covariance pooling networks by iterative matrix square root normalization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 947–955.
- [9] Q. Wang, P. Li, and L. Zhang, "G2DeNet: Global Gaussian distribution embedding network and its application to visual recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2730–2739.
- [10] X. Wei, Y. Zhang, Y. Gong, J. Zhang, and N. Zheng, "Grassmann pooling as compact homogeneous bilinear pooling for fine-grained visual classification," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Sep. 2018, pp. 355–370.
- [11] R. Bro, E. Acar, and T. G. Kolda, "Resolving the sign ambiguity in the singular value decomposition," *J. Chemometrics, A J. Chemometrics Soc.*, vol. 22, no. 2, pp. 135–140, 2008.
- [12] S. H. Lee, D. H. Kim, and B. C. Song, "Self-supervised knowledge distillation using singular value decomposition," in *Proc. Eur. Conf. Comput. Vis. Springer*, 2018, pp. 339–354.
- [13] L. Song, B. Du, L. Zhang, L. Zhang, J. Wu, and X. Li, "Nonlocal patch based T-SVD for image inpainting: Algorithm and error analysis," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 2419–2426.
- [14] T.-Y. Lin and S. Maji, "Improved bilinear pooling with CNNs," 2017, *arXiv:1707.06772*. [Online]. Available: <http://arxiv.org/abs/1707.06772>
- [15] Z. Gao, J. Xie, Q. Wang, and P. Li, "Global second-order pooling convolutional networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 3024–3033.
- [16] M. Lin, Q. Chen, and S. Yan, "Network in network," 2013, *arXiv:1312.4400*. [Online]. Available: <http://arxiv.org/abs/1312.4400>
- [17] X. Zhang and X. Zhang, "Global learnable pooling with enhancing distinctive feature for image classification," *IEEE Access*, vol. 8, pp. 98539–98547, 2020.
- [18] H. Azizpour, A. S. Razavian, J. Sullivan, A. Maki, and S. Carlsson, "From generic to specific deep representations for visual recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2015, pp. 36–45.
- [19] B. Zhang, Q. Zhao, W. Feng, and S. Lyu, "AlphaMEX: A smarter global pooling method for convolutional neural networks," *Neurocomputing*, vol. 321, pp. 36–48, Dec. 2018.
- [20] T.-Y. Lin, A. RoyChowdhury, and S. Maji, "Bilinear CNN models for fine-grained visual recognition," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1449–1457.
- [21] K. Dabov, A. Foi, V. Katkovnik, and K. Egiazarian, "Bm3D image denoising with shape-adaptive principal component analysis," Tech. Rep., 2009.
- [22] L. Hazelhoff, J. Han, and P. H. N. de With, "Video-based fall detection in the home using principal component analysis," in *Proc. Int. Conf. Adv. Concepts Intell. Vis. Syst.*, Springer, 2008, pp. 298–309.
- [23] L. E. Blumenson, "A derivation of n-dimensional spherical coordinates," *Amer. Math. Monthly*, vol. 67, no. 1, pp. 63–66, Jan. 1960. [Online]. Available: <http://www.jstor.org/stable/2308932>
- [24] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
- [25] S. Kullback and R. A. Leibler, "On information and sufficiency," *Ann. Math. Statist.*, vol. 22, no. 1, pp. 79–86, 1951.
- [26] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Citeseer, Tech. Rep., 2009.
- [27] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.
- [28] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. OSDI*, vol. 16, 2016, pp. 265–283.
- [29] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27th Int. Conf. Mach. Learn. (ICML)*, 2010, pp. 807–814.
- [30] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [31] J. Kiefer and J. Wolfowitz, "Stochastic estimation of the maximum of a regression function," *Ann. Math. Statist.*, vol. 23, no. 3, pp. 462–466, Sep. 1952.
- [32] Y. Nesterov, "A method for unconstrained convex minimization problem with the rate of convergence $O(1/k^2)$," in *Proc. Doklady AN USSR*, vol. 269, 1983, pp. 543–547.
- [33] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, Nov. 1987.
- [34] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*. [Online]. Available: <http://arxiv.org/abs/1412.6572>
- [35] A. Kurakin, I. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," 2016, *arXiv:1607.02533*. [Online]. Available: <http://arxiv.org/abs/1607.02533>
- [36] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*. [Online]. Available: <http://arxiv.org/abs/1409.1556>



SEUNGHYUN LEE (Associate Member, IEEE) received the B.S. degree in electronic engineering from Inha University, Incheon, South Korea, in 2017, where he is currently pursuing the Ph.D. degree. His research interests include computer vision and machine learning.



BYUNG CHEOL SONG (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electrical engineering from the Korea Advanced Institute of Science and Technology, Daejeon, South Korea, in 1994, 1996, and 2001, respectively. From 2001 to 2008, he was a Senior Engineer with the Digital Media Research and Development Center, Samsung Electronics Company Ltd., Suwon, South Korea. He joined the Department of Electronic Engineering, Inha University, Incheon, South Korea, in 2008, where he is currently a Professor. His research interests include general areas of image processing and computer vision.

...